🌐 50% Completion Plan – Intent-Aware Intelligent Traffic Routing (eBPF + Kubernetes)

Component Owner: Samarasinghe P.P. – IT22036384
Project ID: 25-26J-434
Specialization: Software Engineering
Date: November 19, 2025

✅ Overview

The goal is to implement **50% of the component**: a real-time traffic routing mechanism that uses **eBPF kernel telemetry** and **intent-driven policies** to reroute service traffic dynamically in a Kubernetes cluster.

🧩 Summary of Component

- **Purpose:** Enable real-time, kernel-level dynamic service-to-service routing.
- **Telemetry Used:** DNS delay, TCP handshake time, packet retransmissions.
- **Decision Engine:** Evaluates real-time metrics against user-defined intents.
- **Action:** Enforces rerouting via in-kernel logic (no sidecars).

📋 Work Breakdown Structure (WBS) for 50% Completion

Phase	Task	Tools/Concepts	Status
1️⃣ Setup & Environment	Set up local test Kubernetes cluster (Minikube/kind + Linux ≥ 5.x kernel)	Minikube, Docker	☐
	Install eBPF toolchain (clang/llvm, bpftool, libbpf-go)	eBPF Dev Tools	☐
	Deploy eBPF DaemonSet from Component 1	Collaborate with teammate	☐
2️⃣ Kernel-Level Telemetry Collection	Attach eBPF probes: `tcp_connect`, `dns_request_start`, etc.	XDP/TC, kprobes	☐
	Log metrics: DNS delay, handshake latency, packet loss	Perf buffers / ring buffer	☐
3️⃣ Routing Intelligence Layer	Design “Intent” model (YAML/JSON policy)	Policy parser / OPA	☐
	Implement routing logic to evaluate metrics vs intent	Golang / Python	☐

Phase	Task	Tools/Concepts	Status
	Create decision tree logic (threshold → reroute)	Rule engine	☐
4️⃣ Enforcement Actions	Implement eBPF reroute logic at socket/XDP level	sockmap / XDP / tc-bpf	☐
	Test: reroute `Service A → Service B2` on poor metrics	Netcat / test pods	☐
5️⃣ Interface Integration	Receive telemetry from Component 1 interface	GRPC / REST	☐
	Send reroute alerts to Component 4	gRPC / gossipsub	☐

📊 Milestone – Deliverables for 50% Completion

- ☐ Working eBPF probes logging **3 key metrics** (DNS delay, TCP latency, retransmits).
- ☐ Basic routing engine that acts on **intent thresholds**.
- ☐ One reroute test case with enforced path change.
- ☐ Integration with low-level interface from Component 1.
- ☐ Live simulation logs/graphs for routing decisions.

🛠️ Recommended Tool Stack

Purpose	Tools
eBPF Programming	`bcc`, `libbpf`, `bpftool`, `clang/llvm`
Telemetry Aggregation	`Prometheus`, `Grafana`, `OpenTelemetry`
Programming	Go / Python / Rust
Cluster Setup	Minikube / Kind / Bare-metal
Traffic Testing	`curl`, `wrk`, `iperf`, `netcat`
Debugging	`tcpdump`, `perf`, `cilium monitor`

🔄 Suggested Weekly Timeline

Week	Goal
Week 1	Environment setup, install eBPF tools, DNS delay probe
Week 2	Complete metric collection (3 metrics)
Week 3	Implement routing intent logic and basic rule parser
Week 4	Implement rerouting and test 1 case

Week	Goal
Week 5	Interface connection with Component 1 + logs ready

> ⚠ Let me know if you want this converted into a Gantt chart, Kanban board, or PDF version.

🔧 Tasks You Can Do Before Component 1 Is Complete

Even if Component 1 (eBPF Daemon Layer) is still in progress, you can parallelly work on the following:

1. ✍️ Define Performance Intents and Routing Policy Models

- Design a simple **YAML or JSON schema** for declaring user intents (e.g., latency < 100ms).
- Define different use cases (e.g., preferred zone, latency sensitivity, failover triggers).

2. 🧠 Develop Your Decision Engine (Logic Layer)

- Create a **policy parser** that reads intents and forms routing rules.
- Simulate routing decision logic using **dummy telemetry data** (you can mock DNS latency or handshake delays).

3. 🧪 Build Test Harness for Routing Simulation

- Develop mock services or pods (`Service A`, `Service B1`, `Service B2`) using Minikube.
- Use **network emulation tools** (like `tc`, `netem`) to introduce latency, packet loss for testing.
- Build shell/Python scripts to simulate flow decisions using your logic layer.

4. 📊 Design Evaluation Metrics and Dashboards

- Define how you will measure success (latency, reroute reaction time, etc.).
- Create Prometheus metrics format or Grafana dashboard templates in advance.

5. 📚 Research & Learn

- Study **eBPF traffic control and redirection patterns**.
- Explore **socket-level vs XDP-level traffic rerouting**.
- Review **Open Policy Agent (OPA)** basics for future intent enforcement.

> By completing these ahead of time, you'll be fully prepared to integrate once Component 1's telemetry interface is ready.